```
    <% end %>
      <td class="text-right"><%= line_item.quantity %></td>
      <td>&times;</td>
      <td class="pr-2">
        <%= line_item.product.title %>
      </td>
      <td class="text-right font-bold">
        <%= number_to_currency(line_item.total_price) %>
      </td>
    </tr>
```

But for email templates, Rails provides a bit more naming magic. If you create multiple templates with the same name but with different content types embedded in their filenames, Rails will send all of them in one email, arranging the content so that the email client can distinguish each.

This means you'll want to either update or delete the plain-text template that Rails provided for the `shipped` notifier.

Testing Email

When we used the generate script to create our order mailer, it automatically constructed a corresponding `order_test.rb` file in the application's `test/mailers` directory. It's pretty straightforward; it simply calls each action and verifies selected portions of the email produced. Because we've tailored the email, let's update the test case to match:

rails7/depot_qa/test/mailers/order_mailer_test.rb

```
    require "test_helper"

    class OrderMailerTest < ActionMailer::TestCase
      test "received" do
»       mail = OrderMailer.received(orders(:one))
»       assert_equal "Pragmatic Store Order Confirmation", mail.subject
»       assert_equal ["dave@example.org"], mail.to
»       assert_equal ["depot@example.com"], mail.from
»       assert_match /1 x Programming Ruby 1.9/, mail.body.encoded
      end

      test "shipped" do
```